Lavorare con i Dati Usando l'AI

Lezione 2 — Dal vector database all'agente

Chi sono

Federico Romeo

- AI Engineer presso **aidea**
- Docente per **Boolean**



Dove eravamo rimasti

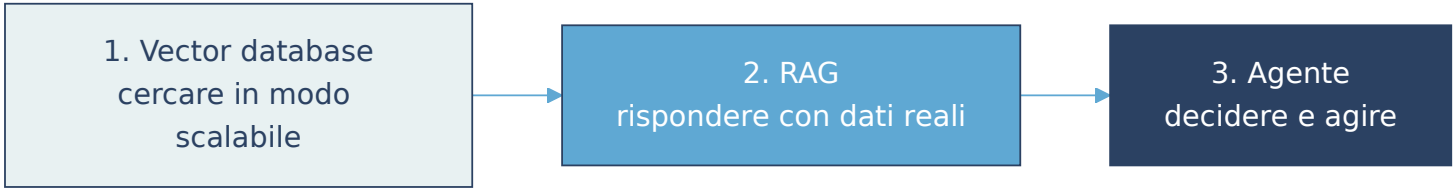
La volta scorsa

- Abbiamo usato un LLM per trasformare **testo grezzo di ricette** in **JSON strutturato**
- Abbiamo visto che la **ricerca per keyword** confronta parole, non significato
- Abbiamo trasformato 1.000 ricette in **embeddings**: un vettore di 1.536 numeri per ricetta
- Con la **cosine similarity** abbiamo cercato per significato

Query	Keyword	Semantica
`"dish with marinara"`	solo le ricette con la parola "marinara"	anche quelle con salsa di pomodoro
`"quick weeknight dinner"`	**zero risultati**	piatti veloci e semplici

Il piano di oggi

Prendiamo la ricerca semantica e la facciamo diventare un **agente** che agisce nel mondo reale.



Il filo conduttore resta lo stesso: **le nostre 1.000 ricette**.

Perché ci serve un vector database?

NumPy funziona, ma...

La volta scorsa abbiamo calcolato la cosine similarity con NumPy: su 1.000 ricette va benissimo.

Ma con **100.000 ricette**? O **1 milione**?

- Ogni query confronta **tutti** i vettori → O(n)
- Tutti i vettori devono stare **in memoria**
- Nessuna persistenza: gli embedding si perdono quando il notebook si riavvia
- Nessun filtro (es. "solo ricette vegetariane")

Un **vector database** risolve tutto questo:

- Algoritmi di **Approximate Nearest Neighbor** (ANN) → veloce anche su larga scala
- **Storage persistente**: i dati sopravvivono ai riavvii
- **Filtri sui metadati**: combina ricerca semantica e filtri

Come fa a essere veloce?

Invece di confrontare la query con **ogni** vettore, il database costruisce un **indice**.

- L'indice più usato si chiama **HNSW**: un grafo in cui ogni vettore è collegato ai suoi vicini
- La ricerca parte da un punto e **salta di vicino in vicino**, sempre verso la query
- Pochi salti invece di milioni di confronti

Il prezzo: è **approssimato**. Ogni tanto può perdere il vicino esatto.

In cambio risponde in **millisecondi** su milioni di vettori: per la ricerca semantica è un ottimo affare.

Qdrant

Useremo **Qdrant**, un vector database open-source.

- Si avvia in locale con un singolo `docker compose up`
- Ha una **dashboard web** su `localhost:6333/dashboard`
- Il client Python è semplice e intuitivo

Tre concetti:

Qdrant	Equivalente "classico"	Nel nostro caso
Collection	una tabella	`recipes`
Point	una riga	una ricetta
Vector	la colonna su cui si cerca	l'embedding della ricetta
Payload	le altre colonne	titolo, ingredienti, istruzioni

Proviamolo

notebook `session2/segment_2.ipynb` — Qdrant

Dalla ricerca all'assistente

E se un LLM rispondesse usando le nostre ricette?

Ora abbiamo una funzione che prende una query e restituisce le ricette più rilevanti.

1. L'utente chiede: *"Cosa posso cucinare con il pollo avanzato?"*
2. Trasformiamo la domanda in embedding e cerchiamo in Qdrant
3. Prendiamo le **5 ricette più rilevanti**
4. Le passiamo all'LLM come **contesto**, insieme alla domanda

Questo pattern si chiama **RAG**, Retrieval-Augmented Generation.

L'LLM non inventa: prima **recupera** dati reali, poi **genera** una risposta basata su quei dati.

L'architettura



Qui siamo **noi** a decidere quando cercare: a ogni domanda, sempre.

Costruiamolo

notebook `session2/segment_3.ipynb` — RAG

Il limite del nostro assistente

Il problema: nessun contesto

Nel RAG che abbiamo appena costruito, **eravamo noi** a chiamare `retrieve_recipes()` .

Ogni query era **indipendente**: il sistema non aveva memoria.

- Turno 1: *"Cerco una ricetta con il pesce"* → funziona!
- Turno 2: *"Dammene un'altra"* → ???

"Dammene un'altra" viene embeddata **alla lettera**, e il vector database non sa di cosa stessimo parlando.

Vediamolo in pratica.

Vediamolo

notebook `session3/segment_1.ipynb` — il limite del loop naive

Function Calling

E se l'LLM potesse usare strumenti?

Finora l'LLM riceveva testo e rispondeva con testo.

Con il **function calling**, l'LLM può fare qualcosa di nuovo:

- Vede una lista di **tool disponibili** (funzioni che può chiamare)
- Analizza la richiesta dell'utente
- **Decide da solo** se e quale tool usare
- Restituisce una **richiesta strutturata** (JSON) per eseguire il tool

L'LLM **non esegue** il codice: chiede a noi di farlo. Noi eseguiamo, restituiamo il risultato, e l'LLM genera la risposta finale.

Il loop agentico



Il contesto della conversazione viene **mantenuto**: "dammene un'altra" ora funziona.

È la stessa ricerca di prima, ma ora a decidere **quando** e **cosa** cercare è l'LLM.

Proviamolo

notebook `session3/segment_2.ipynb` — function calling

Dall'assistente all'agente

Ok, cerca ricette. Ma se potesse anche *agire*?

Finora il nostro agente può solo **cercare** ricette e **rispondere**.

E se potesse anche **fare qualcosa nel mondo reale**?

- Trovare una ricetta
- Estrarne gli ingredienti
- **Creare una lista della spesa** su Trello, automaticamente

Basta aggiungere un secondo tool. L'agente **decide da solo** la sequenza:

1. Cerca la ricetta con `retrieve_recipes`
2. Aggiunge gli ingredienti con `add_to_shopping_list`

L'architettura



Costruiamolo

notebook `session3/segment_3.ipynb` — integrazione Trello

Cosa abbiamo costruito

Da testo grezzo ad agente autonomo

La volta scorsa: LLM e ricerca semantica

- Testo grezzo → dati strutturati (JSON)
- Testo → vettori → ricerca per significato

Oggi: dal database all'agente

- Vector database: la ricerca semantica in modo scalabile
- RAG: risposte basate su dati reali
- Function calling: l'LLM decide quali tool usare
- Integrazione con il mondo reale (Trello): da "risponde" ad "agisce"

Il pattern è sempre lo stesso: **dare all'LLM gli strumenti giusti e lasciarlo ragionare**.

Grazie!

Domande?